Universidad de Carabobo

Facultad de Ciencias y Tecnología

Arquitectura del Computador

Práctica de Laboratorio 3: E/S Mapeada en Memoria (MMIO) en MIPS32

Autores:
José Álvarez — C.I.: 31389258
José Núñez — C.I.: 31581050

1 de septiembre de 2026

Preguntas del Informe

1. Explique cómo se organiza la memoria cuando un sistema memory-mapped I/O. ¿En qué región de memoria se suelen mapear los dispositivos? ¿Qué implicaciones tiene para las instrucciones lw y sw?

En un sistema con *memory-mapped I/O* (MMIO), el espacio de direcciones de memoria se comparte de forma unificada entre la memoria principal (RAM) y los registros de control/estado de los periféricos de hardware. Cada dispositivo de E/S ocupa un rango específico y exclusivo de direcciones físicas dentro de este mapa global. Habitualmente, los dispositivos se mapean en regiones reservadas, ya sea en las zonas superiores del mapa de memoria o en segmentos especializados de E/S definidos por la arquitectura. La principal implicación de este enfoque es que no se requieren instrucciones especiales de entrada/salida; el procesador utiliza exactamente las mismas instrucciones estándar de carga (`lw`) y almacenamiento (`sw`) empleadas para acceder a la RAM tradicional.

2. ¿Cuál es la principal diferencia entre memory-mapped I/O y la entrada/salida por puertos? ¿Qué ventajas y desventajas tiene cada enfoque? ¿Por qué MIPS32 utiliza principalmente memory-mapped I/O?

La diferencia fundamental radica en que *memory-mapped I/O* comparte un único espacio de direcciones físicas tanto para la memoria como para los dispositivos, mientras que la E/S por puertos (*port-mapped I/O* o E/S aislada) mantiene un espacio de direcciones de E/S completamente independiente de la memoria principal.

- **MMIO:** *Ventajas:* Simplifica el diseño del software al emplear instrucciones comunes de memoria y aprovecha todos los modos de direccionamiento disponibles. *Desventajas:* Consume una porción del espacio de direcciones que de otro modo estaría disponible exclusivamente para la memoria RAM.
- **E/S por puertos:** *Ventajas:* Todo el espacio de direcciones de la CPU se reserva íntegramente para la memoria RAM sin interferencias. *Desventajas:* Requiere instrucciones dedicadas (como `IN` y `OUT` en x86) y circuitería de decodificación de bus adicional.

MIPS32 utiliza principalmente MMIO porque sigue fielmente la filosofía RISC de mantener un conjunto de instrucciones limpio, reducido y uniforme, evitando la sobrecarga de añadir opcodes e instrucciones hardware adicionales para manejar periféricos.

3. En un sistema con memory-mapped I/O, ¿Qué problemas pueden surgir si dos dispositivos usan direcciones solapadas? ¿Cómo se evita este conflicto?

Si dos dispositivos distintos utilizan rangos de direcciones que se solapan en el mapa físico, se producen graves conflictos en el bus del sistema. Al activarse una señal de lectura o escritura en esa dirección, ambos periféricos responderán simultáneamente, corrompiendo el bus de datos con señales eléctricas superpuestas y provocando fallos impredecibles en el sistema o bloqueos de la CPU. Este conflicto se evita implementando un decodificador de direcciones estricto y jerárquico en la circuitería del bus, el cual garantiza que cada bloque de memoria o registro de E/S pertenezca de forma unívoca y exclusiva a un único periférico.

4. ¿Por qué se considera que el memory-mapped I/O simplifica el diseño del conjunto de instrucciones de un procesador? ¿Qué tipo de instrucciones adicionales serían necesarias si se usara E/S por puertos?

Se considera que simplifica el conjunto de instrucciones porque el diseñador del procesador no necesita añadir comandos especializados para interactuar con el exterior; cualquier instrucción que pueda leer o escribir en la memoria (como `lw`, `sw`, `lb`, `sb`) sirve de manera directa para comunicarse con los periféricos. Si se optara por E/S por puertos, la arquitectura se vería obligada a incorporar instrucciones dedicadas (ej. `IN`, `OUT`, `INS`, `OUTS`), además de líneas de control específicas en el bus del procesador (por ejemplo, señales de solicitud de E/S o *I/O Request*).

5. ¿Qué ocurre a nivel del bus de datos y direcciones cuando el procesador accede a una dirección de memoria que corresponde a un dispositivo? ¿Cómo sabe el hardware que debe acceder a un periférico en lugar de la RAM?

Cuando el procesador ejecuta una instrucción sobre una dirección mapeada, coloca dicha dirección en el bus de direcciones y activa las líneas de control correspondientes (como lectura `RD` o escritura `WR`). En caso de una escritura, el dato se coloca simultáneamente en el bus de datos. El hardware sabe que debe atender a un periférico y no a la RAM gracias al decodificador de direcciones del sistema, el cual supervisa permanentemente el bus de direcciones y genera una señal exclusiva de selección de chip (*Chip Select*) destinada al módulo de E/S cuando la dirección se encuentra dentro de su rango asignado, inhabilitando al mismo tiempo a los bancos de memoria principal.

6. ¿Es posible que un programa normal (sin privilegios) acceda a un dispositivo mapeado en memoria? ¿Qué mecanismos de protección existen para evitar accesos no autorizados?

En un sistema operativo moderno con protección de hardware, un programa de usuario estándar (sin privilegios) **no** tiene permitido acceder directamente a las direcciones físicas donde están mapeados los dispositivos de E/S. Para evitar accesos no autorizados que comprometan la estabilidad del sistema, se implementan mecanismos como el **modo dual de operación** (Modo Usuario y Modo Supervisor/Kernel) administrado por la MMU (*Memory Management Unit*). Las páginas de memoria correspondientes al espacio de E/S están marcadas con privilegios de nivel de kernel; si una aplicación de usuario intenta leerlas o escribirlas directamente, la CPU genera una excepción por violación de privilegios o segmento, y el sistema operativo intercepta y termina el proceso de forma segura.

7. ¿Qué técnicas se pueden emplear para evitar esperas activas innecesarias al interactuar con dispositivos?

Para evitar el consumo ineficiente de ciclos de reloj que provoca la espera activa (*polling*), se emplean principalmente dos técnicas:

1. **Interrupciones por hardware:** El dispositivo notifica de manera asíncrona a la CPU mediante una señal eléctrica de interrupción cuando su tarea ha finalizado o hay datos listos, permitiendo que el procesador ejecute otras tareas productivas mientras tanto.

2. **Acceso Directo a Memoria (DMA):** Un controlador especializado gestiona las transferencias masivas de datos entre los periféricos y la memoria principal de forma autónoma, liberando por completo a la CPU de la tarea hasta que la operación completa ha concluido.

8. Análisis y Discusión de los Resultados

Durante el desarrollo y prueba de los procedimientos en el entorno MARS, se comprobó el correcto funcionamiento de los mecanismos de *memory-mapped I/O* (MMIO). Al implementar bucles de espera activa (*polling*), el procesador permaneció en un ciclo continuo de lectura sobre los registros de estado (`LuzEstado` y `TensionEstado`) hasta que se simuló la modificación de dichos valores directamente en el segmento de datos.

Específicamente, en el primer ejercicio se verificó que la rutina responde adecuadamente tanto al código de éxito (valor 1) como a la señal de error de hardware (valor -1), permitiendo la extracción correcta del valor de luminosidad almacenado. Del mismo modo, en el segundo ejercicio, el procedimiento `controlador_tension` logró inicializar el módulo escribiendo el bit de control y recolectar de forma exacta las tensiones sistólica y diastólica en los registros de retorno `$v0` y `$v1`, respectivamente. Estos resultados demuestran la viabilidad y sencillez de coordinar periféricos virtuales simulando celdas de memoria estándar mediante el uso de instrucciones de carga y almacenamiento (`lw` y `sw`), cumpliendo a cabalidad con los objetivos planteados en la práctica.